

Deep Learning Architecture Reference

Section 1: BERT-large Model Specifications

BERT-large is a transformer-based model released by Google in 2018. BERT-large has 340 million parameters, 24 encoder layers, 16 attention heads, and a hidden size of 1024. The model was pre-trained on BooksCorpus and English Wikipedia using two objectives: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). BERT-large achieves state-of-the-art performance on the GLUE benchmark with a score of 80.5.

Section 2: GPT-4 Technical Details

GPT-4 is a large multimodal model developed by OpenAI, released in March 2023. GPT-4 supports both text and image inputs with text outputs. The model uses Reinforcement Learning from Human Feedback (RLHF) for alignment. GPT-4 scored in the 90th percentile on the Uniform Bar Exam and achieved 163 on the LSAT. The context window of GPT-4-turbo supports up to 128,000 tokens.

Section 3: Error Codes and API Reference

Common API error codes: HTTP 429 means rate limit exceeded. HTTP 503 means service unavailable. Error code `CONTEXT_LENGTH_EXCEEDED` is raised when the input exceeds the maximum token limit. The embedding model `text-embedding-ada-002` has a maximum input of 8191 tokens and outputs vectors of 1536 dimensions. For ChromaDB, the default distance metric is L2 (Euclidean distance). Use cosine similarity by setting `metadata={'hnsw:space': 'cosine'}`.

Section 4: How the Transformer Learns to Pay Attention

When processing a sequence of words, the model needs to understand which parts of the input are relevant to each other. It does this by computing a score between every pair of positions in the sequence. Positions that are strongly related receive a high score and influence each other more during processing. This mechanism allows the model to capture long-range dependencies that older sequential architectures like RNNs struggled with. The key insight is that relevance is computed dynamically based on the content of the input, not fixed by position.

Section 5: Why Language Models Sometimes Make Things Up

Large language models generate text by predicting the most probable next word given everything that came before. The model has no internal fact-checker or access to external knowledge at inference time. When asked about something rare or outside its training data, it continues producing fluent text by drawing on patterns — even if those patterns lead to incorrect conclusions. This tendency to produce confident but false information is called hallucination. Retrieval-augmented generation addresses this by grounding each response in retrieved documents before generating an answer.

Section 6: The Intuition Behind Word Embeddings

Imagine mapping every word in the English language to a point in a very high-dimensional space. Words that are used in similar contexts end up placed near each other in this space. So 'king' and 'queen' are close together. 'Dog' and 'puppy' are close together. But 'dog' and 'democracy' are far apart. The interesting property is that relationships between concepts are encoded as directions: the direction from 'man' to 'woman' is roughly the same as the direction from 'king' to 'queen'. This geometric structure emerges entirely from training on large text corpora, not from any human-defined rules about word meaning.

Section 7: Retrieval-Augmented Generation (RAG) Overview

RAG combines a retrieval system with a generative language model. At query time, the system first searches a document store for relevant passages, then passes those passages as context to the language model. This approach reduces hallucination because the model is anchored to retrieved facts rather than relying purely on memorized training data. RAG is particularly useful for enterprise applications where the knowledge base changes frequently and retraining the model is not practical.

Section 8: Chunking Strategies for RAG Systems

Chunking is the process of splitting documents into smaller pieces before indexing. Fixed-size chunking splits text every N characters regardless of sentence boundaries. Recursive character text splitting tries to split at natural boundaries like paragraphs, then sentences, then words, only falling back to character-level splits if needed. Semantic chunking uses embedding similarity to detect topic shifts and split there. Chunk overlap (typically 10-20% of chunk size) helps preserve context across chunk boundaries. A chunk size of 500-1000 characters works well for most RAG applications.

Section 9: Vector Databases Compared

ChromaDB is an open-source, embedded vector store ideal for local development. It runs in-process with no separate server needed and persists data to disk. Qdrant is a production-grade vector database

written in Rust, offering filtering, payload indexing, and horizontal scaling. Pinecone is a fully managed cloud vector database with a generous free tier. FAISS (Facebook AI Similarity Search) is a library optimised for billion-scale nearest neighbour search, commonly used when you need maximum query speed on-premises. For this project, ChromaDB is recommended for local development and Qdrant for production.